

SIMATS ENGINEERING

Object Oriented Analysis and Design (CSA11)

Assignment Answer Script

Object-Oriented Analysis and UML-Based Design of an AI-Enabled Smart Campus Management System (AISCAMS)

Assignment Title	Object-Oriented Analysis and UML-Based Design of an AI-Enabled Smart Campus Management System (AISCAMS)
Course Outcome(s) CO	CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices. CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system. CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education SDG 9 – Industry, Innovation and Infrastructure SDG 11 – Sustainable Cities and Communities

Assignment Weightage: 100%

QUESTIONS: 1 Total Marks: 100

Assignment Weightage: 100%

Annexure A
Assignment

Code of Conduct:

I P.Srinivas(192320044) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**P.SRINIVAS**

1.1 Domain Overview

The AI-Enabled Smart Campus Management System (AISCAMS) is a centralized, role-based platform intended to unify academic, administrative, infrastructure, and student-support services of a college campus. It replaces fragmented manual and semi-digital processes (paper schedules, physical booking registers, verbal maintenance complaints) with an integrated software system accessed by five categories of human actors — students, faculty, administrators, security personnel, and maintenance staff — and supported by two system-level actors: an AI Recommendation Engine and a network of IoT-enabled devices.

The core problem is one of coordination: multiple stakeholders each need timely, accurate information about shared campus resources (classrooms, laboratories, parking, equipment) and shared processes (attendance, service requests, events). Without a common platform, resource allocation is inefficient, maintenance issues go unnoticed, and decision-makers lack real-time visibility into infrastructure usage.

1.2 Key Functional Requirements

- Role-based authentication and access control for five user categories.
- Academic schedule management and classroom/facility booking with conflict detection.
- AI-driven recommendation of suitable classrooms/facilities based on availability, capacity, and historical usage patterns.
- Attendance and class-activity management for faculty.
- Campus event creation and registration.
- Service request submission, assignment, status tracking, and resolution for maintenance issues.
- Real-time infrastructure monitoring via IoT sensors (occupancy, environmental conditions, equipment/device status).
- Notification/alerting to relevant users on status changes (booking confirmed, request resolved, task assigned).
- Administrative reporting and analytics on usage, occupancy, and service performance.
- Parking and campus access/security monitoring.

1.3 Key Non-Functional Requirements

- Scalability — the system must support a growing number of concurrent users and IoT devices without redesign.
- Maintainability — modular packages and loosely coupled components so that individual services can evolve independently.
- Reliability & Availability — critical services (booking, security monitoring) must remain available during peak campus hours.
- Security — role-based access control, encrypted credentials, and audit logging of administrative actions.
- Usability — a consistent role-based interface across web and mobile clients.
- Real-time responsiveness — IoT data and notifications should reach the relevant actor with minimal latency.

1.4 Stakeholders

Primary stakeholders are the students and faculty who consume campus services daily; administrators who govern policy, accounts, and reporting; security personnel who safeguard the campus; and maintenance staff who keep infrastructure operational. Secondary stakeholders include the institution's management (interested in utilisation reports) and prospective students/visitors interacting with public-facing services such as event listings.

2. Actor Identification

Actors were identified by examining every noun phrase in the problem statement that initiates interaction with the system or is external to the system boundary but exchanges information with it. Both human (primary) actors and system-level (supporting) actors are included, following standard use-case actor identification practice.

Actor	Type	Description / Role in the System
Student	Primary (Human)	Views schedules, books facilities, raises and tracks service requests, registers for events.
Faculty	Primary (Human)	Manages attendance, class materials, and requests classrooms for teaching activities.
Administrator	Primary (Human)	Manages user accounts, system configuration, and generates institutional reports.
Security Personnel	Primary (Human)	Monitors campus access/security and manages parking areas.
Maintenance Staff	Primary (Human)	Receives assigned maintenance tasks and updates repair status.
AI Recommendation Engine	Supporting (System)	Recommends facilities/resources and analyses usage patterns; invoked by other use cases.
IoT Sensor Network	Supporting (System)	Streams occupancy, environmental, and equipment-status data into the platform.
Notification Service	Supporting (External System)	Delivers email/SMS/push alerts triggered by system events (e.g., booking confirmation).

3. Use Case Descriptions

Fully-dressed descriptions are provided for the three most design-critical use cases; the remaining use cases are summarised in the table that follows.

3.1 Use Case: Book Classroom / Facility

Field	Description
Use Case ID	UC-03
Actors	Student (primary), Faculty (primary), AI Recommendation Engine (supporting)
Description	Allows a student or faculty member to search for, receive an AI-based recommendation for, and reserve a classroom or campus facility for a specified time slot.
Pre-conditions	Actor is authenticated; requested time slot is within the academic calendar.
Main Flow	1. Actor selects 'Book Facility'. 2. Actor specifies facility type, capacity, and desired time slot. 3. System (via AI Recommendation Engine) suggests the best-matching available facility. 4. Actor confirms the recommended facility or chooses an alternative. 5. System creates a Booking record and reserves the Facility. 6. System sends a confirmation notification to the actor.
Alternate Flow	3a. If no facility matches the criteria, the system widens the search radius (nearby buildings) and re-recommends. 5a. If a scheduling conflict is detected at creation time, the system rejects the booking and returns to step 2.
Post-conditions	A confirmed Booking exists and the Facility's availability status is updated.
Includes / Extends	«include» Get AI Facility Recommendation

3.2 Use Case: Raise Service Request

Field	Description
-------	-------------

Use Case ID	UC-06
Actors	Student, Faculty (primary); Maintenance Staff, Notification Service (supporting)
Description	Enables any campus user to report an infrastructure or facility issue (e.g., broken projector, malfunctioning equipment) and track its resolution.
Pre-conditions	Actor is authenticated and associated with the facility/location being reported.
Main Flow	1. Actor selects 'Raise Service Request' and enters category, description, and priority. 2. System creates a ServiceRequest with status SUBMITTED. 3. System creates a corresponding MaintenanceTask and assigns available Maintenance Staff. 4. Notification Service alerts the assigned staff. 5. Maintenance staff updates status to IN_PROGRESS and later RESOLVED. 6. System notifies the requester of resolution.
Alternate Flow	3a. If no staff with matching specialization is available, the request is queued and escalated to the Administrator. 5a. If additional parts/approval are required, status moves to ON_HOLD until resolved.
Post-conditions	ServiceRequest and MaintenanceTask reach a terminal state (RESOLVED/CLOSED) with a full status history.
Includes / Extends	«extend» Track Request Status; «include» Update Repair/Maintenance Status

3.3 Use Case: Get AI Facility Recommendation

Field	Description
Use Case ID	UC-10
Actors	AI Recommendation Engine (primary/system); Student, Faculty (secondary, as requesters)
Description	Analyses facility availability, capacity, and historical usage/occupancy patterns (fed by IoT data) to recommend the most suitable classroom, laboratory, or resource for a given request.
Pre-conditions	At least one Facility exists with a known availability status; IoT occupancy data (where available) is up to date.
Main Flow	1. Booking or classroom-request use case invokes the engine with selection criteria. 2. Engine filters facilities by availability and capacity. 3. Engine ranks candidates using historical usage/occupancy pattern data. 4. Engine returns the top-ranked facility to the calling use case.
Post-conditions	A ranked recommendation (or empty result) is returned to the invoking use case.
Includes / Extends	Included by: Book Classroom/Facility, Request Classroom for Class

3.4 Summary of Remaining Use Cases

ID	Use Case	Primary Actor(s)	Brief Description
UC-01	Login / Authenticate	All actors	Verifies credentials and grants role-based access.
UC-02	View Academic Schedule	Student, Faculty	Displays personalised class/exam timetable.
UC-04	Register for Campus Event	Student	Allows sign-up for institution events.
UC-05	Track Request Status	Student, Faculty	Shows real-time status of a submitted service request.
UC-07	Manage Attendance	Faculty	Records and reports student attendance per session.
UC-08	Upload Class Material	Faculty	Publishes digital resources for a course.
UC-09	Request Classroom for Class	Faculty	Reserves a room for a scheduled teaching session.
UC-11	Monitor Infrastructure (IoT)	Administrator, AI Engine	Displays live occupancy/environment/equipment status.
UC-12	Manage User Accounts	Administrator	Creates, edits, deactivates user accounts and roles.
UC-13	Generate Reports	Administrator	Produces utilisation, performance, and service reports.

UC-14	Monitor Campus Access / Security	Security Personnel	Tracks entry/exit and raises incident logs.
UC-15	Manage Parking	Security Personnel, Student	Tracks parking slot occupancy and allocation.
UC-16	Update Repair/Maintenance Status	Maintenance Staff	Updates the lifecycle state of an assigned task.
UC-17	Receive Notifications	All actors	Delivers system-generated alerts to the actor.

4. Use Case Diagram

The diagram below shows the AISCAMS system boundary, all identified actors, and their associated use cases, including «include» and «extend» relationships.

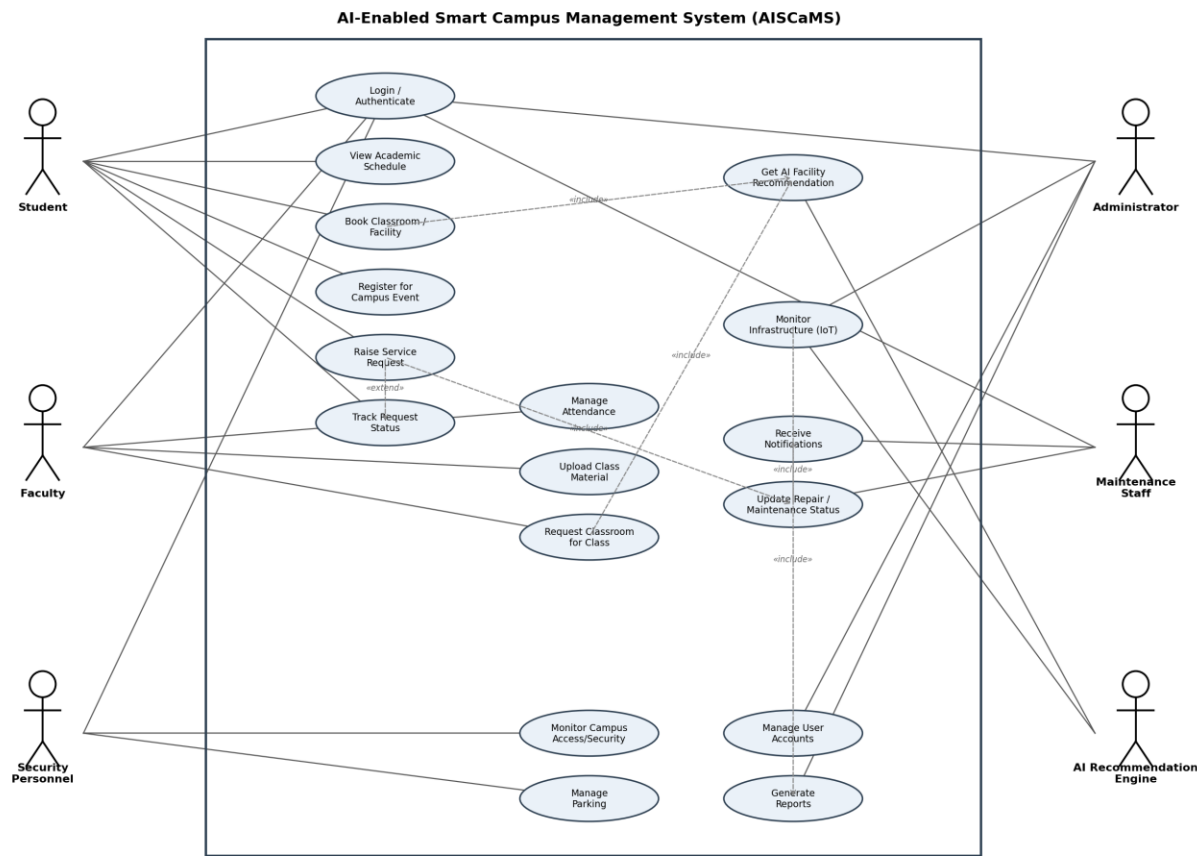


Figure 4.1 — Use Case Diagram of AISCAMS

5. Object and Class Identification

Candidate classes were identified using noun-phrase analysis of the problem statement, then filtered and classified using the Entity–Boundary–Control (Jacobson) approach: Entity classes model persistent domain data, Boundary classes model actor-facing interfaces, and Control classes coordinate use-case logic between them.

Class	Type	Justification
User (+ subclasses)	Entity	Represents any authenticated actor; abstract base for role-specific behaviour (inheritance).

Student, Faculty, Administrator, SecurityPersonnel, MaintenanceStaff	Entity	Role-specific specialisations of User identified directly from the actor list.
Course, AcademicSchedule, Attendance	Entity	Nouns from 'academic schedules' and 'attendance' in the scenario.
Facility (+ Classroom, Laboratory, ParkingArea)	Entity	Nouns from 'classroom bookings', 'campus facilities', 'parking areas'.
Booking	Entity	Represents the reservation transaction between a User and a Facility.
Event	Entity	From 'events' in the student-facing requirement.
ServiceRequest, MaintenanceTask	Entity	From 'service requests' and 'maintenance personnel ... update repair status'.
IoTDevice (+ SmartSensor)	Entity	From 'IoT-enabled devices ... room occupancy, environmental conditions, equipment status'.
AIRecommendationEngine	Control	From 'intelligent recommendations to suggest suitable classrooms/facilities'; coordinates recommendation logic.
NotificationService	Control	Inferred supporting class coordinating outbound alerts across multiple use cases.
Report	Entity	From administrator reporting/analytics responsibilities.
BookingController, ServiceRequestController, UserController	Boundary/Control	Mediate between the UI (boundary) and entity classes for each major use case.
WebPortalUI / MobileAppUI / AdminDashboardUI	Boundary	Role-specific presentation surfaces through which actors interact with the system.

Attributes and operations for each entity class were derived directly from the responsibilities described in the scenario (e.g., a Booking must know its time window and status because the scenario requires conflict-free scheduling; a ServiceRequest must know its category and priority because maintenance staff must triage work).

6. Class Diagram

The class diagram formalises the classes above along with their attributes, operations, and relationships — generalisation (User and Facility hierarchies), composition (a Facility owns the IoTDevice sensors installed in it), aggregation (a Course aggregates its AcademicSchedule), plain association (e.g., Student creates Booking), and dependency (e.g., Booking «uses» AIRecommendationEngine).

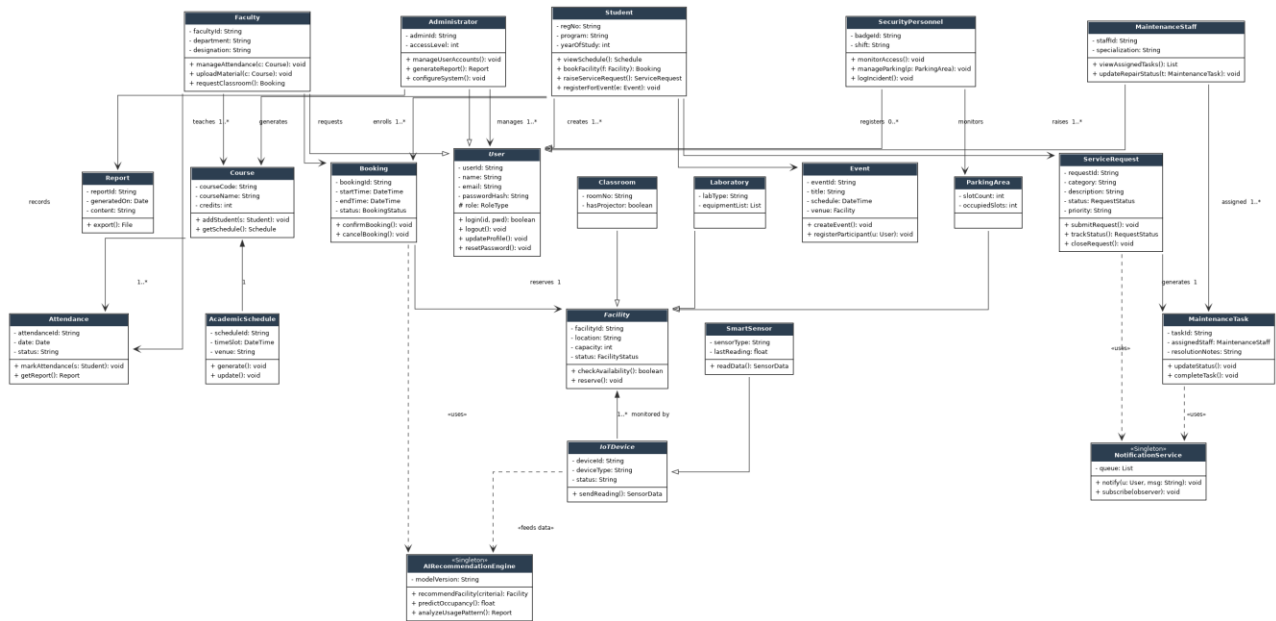


Figure 6.1 — Class Diagram of AISCams

6.1 Notes on Relationships

- Generalisation: Student, Faculty, Administrator, SecurityPersonnel, and MaintenanceStaff inherit common identity/authentication behaviour from the abstract User class (supports polymorphism — e.g., a single login() call behaves per role).
- Composition: IoTDevice instances are part-of a Facility and cannot meaningfully exist without it — modelled with a filled diamond.
- Aggregation: AcademicSchedule is aggregated by Course — a schedule can, in principle, be re-used/viewed independently of a single course instance.
- Dependency: Booking and ServiceRequest depend on (but are not owned by) AIRRecommendationEngine and NotificationService respectively, reflecting a 'uses-a-service' relationship rather than structural ownership.

7. Sequence / Interaction Diagrams

Two representative interaction scenarios are modelled, both directly traceable to the use cases and classes above.

7.1 Sequence Diagram: Book Classroom / Facility

sd Book Classroom / Facility

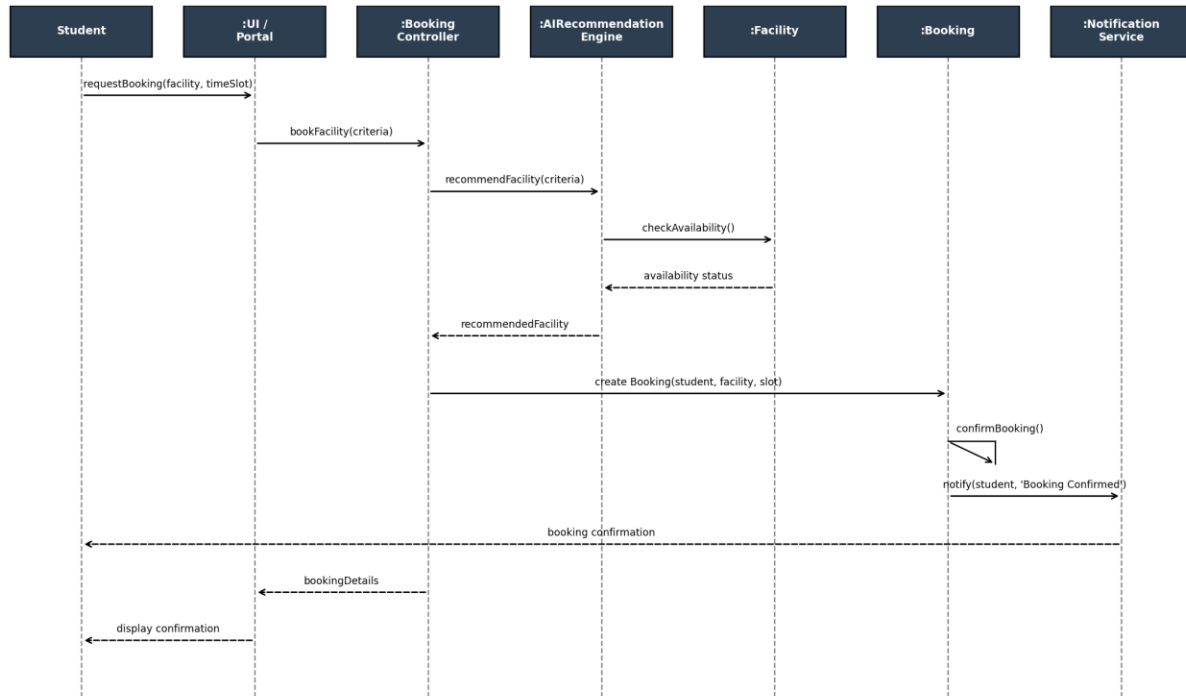


Figure 7.1 — Sequence Diagram for Book Classroom/Facility (UC-03)

This scenario shows the BookingController coordinating with the AIRecommendationEngine to select a Facility, then creating and confirming a Booking, followed by an asynchronous notification back to the Student — directly matching the operations declared on Booking, Facility, and AIRecommendationEngine in the class diagram.

7.2 Sequence Diagram: Raise & Resolve Maintenance Service Request

sd Raise & Resolve Maintenance Service Request

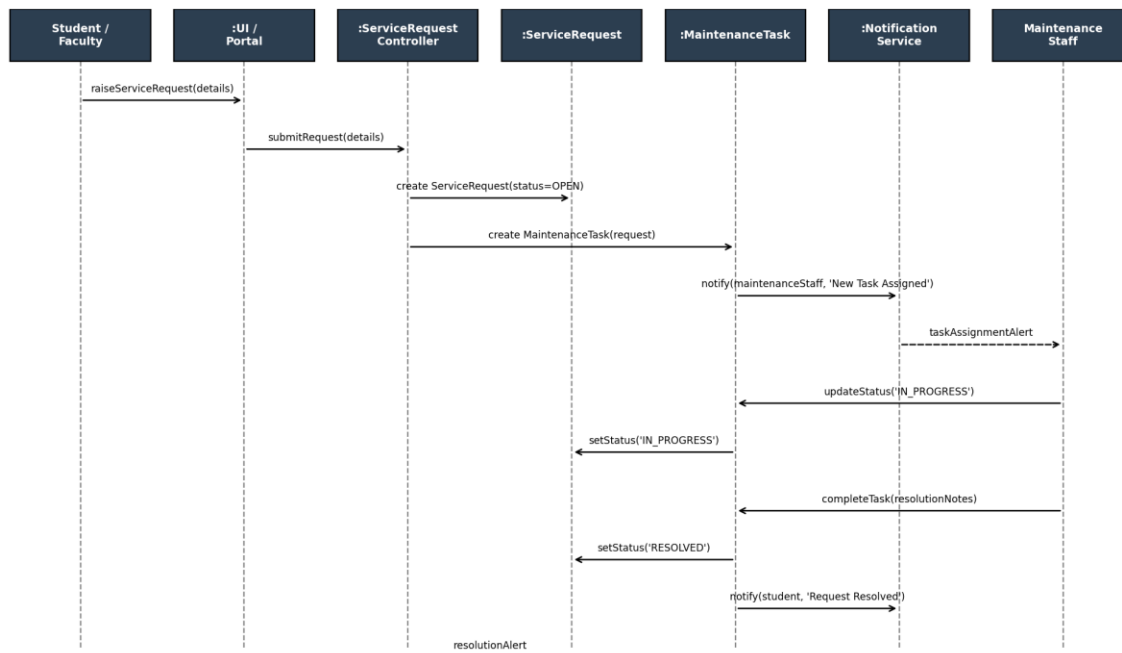


Figure 7.2 — Sequence Diagram for Raise Service Request (UC-06)

This scenario traces the full lifecycle of a request from submission through assignment, progress update, and resolution, showing how `ServiceRequest` and `MaintenanceTask` collaborate with `NotificationService` — consistent with the state transitions defined in Section 9.

8. Activity Diagram

The activity diagram models the control flow of the Classroom Booking process, including the AI-recommendation decision branch and conflict-resolution loop.

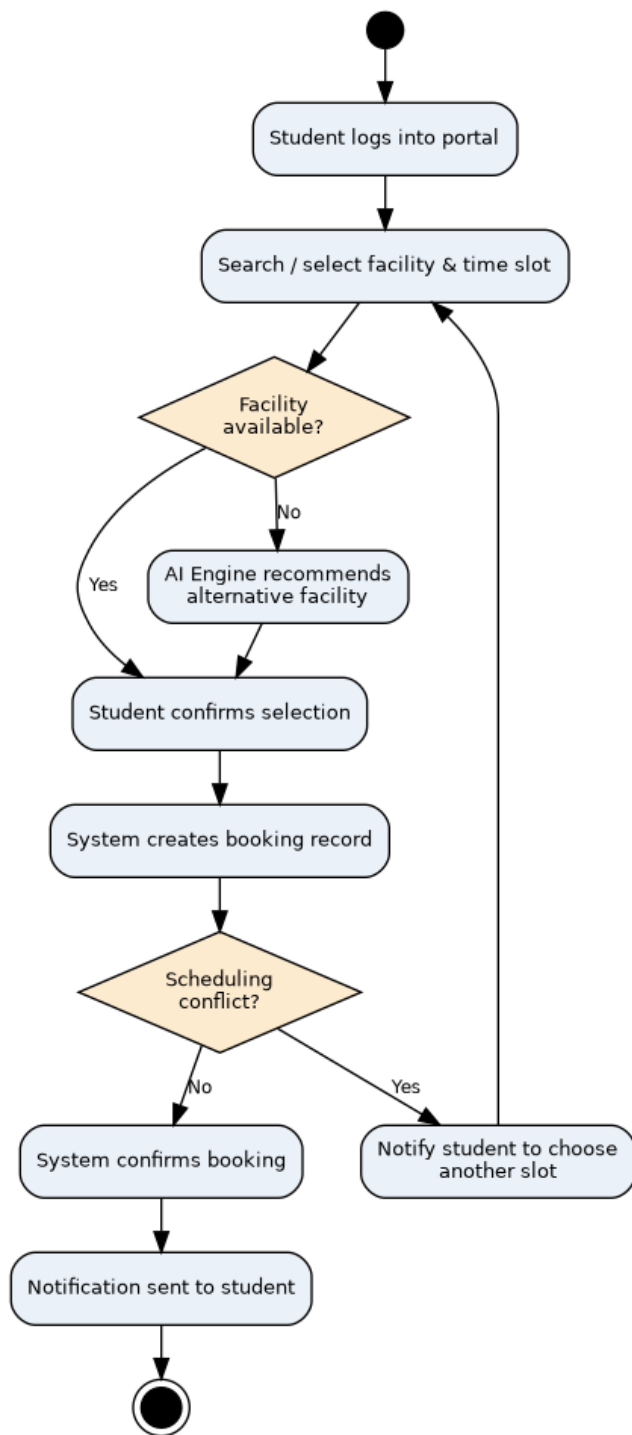


Figure 8.1 — Activity Diagram: Classroom/Facility Booking Process

9. State Transition Diagram

The lifecycle of a `ServiceRequest` / `MaintenanceTask` is modelled as a state machine. This directly governs the operations `updateStatus()`, `completeTask()`, and `closeRequest()` declared in the Class Diagram, and matches the flow shown in the maintenance-request sequence diagram.

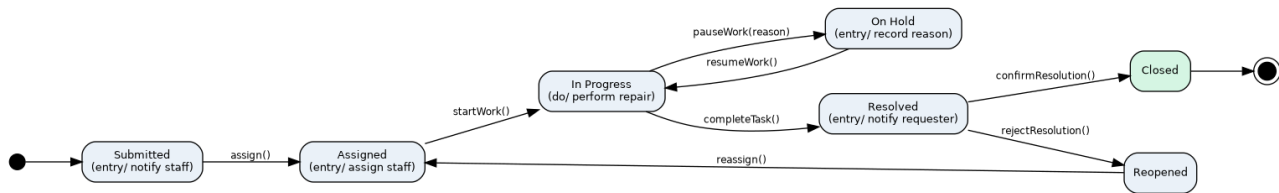


Figure 9.1 — State Transition Diagram for ServiceRequest / MaintenanceTask

10. Package Diagram

The system is organised into layered packages to promote high cohesion within a layer and low coupling across layers, following a layered (N-tier) architectural style suited to a Model-Driven, service-oriented smart-campus platform.

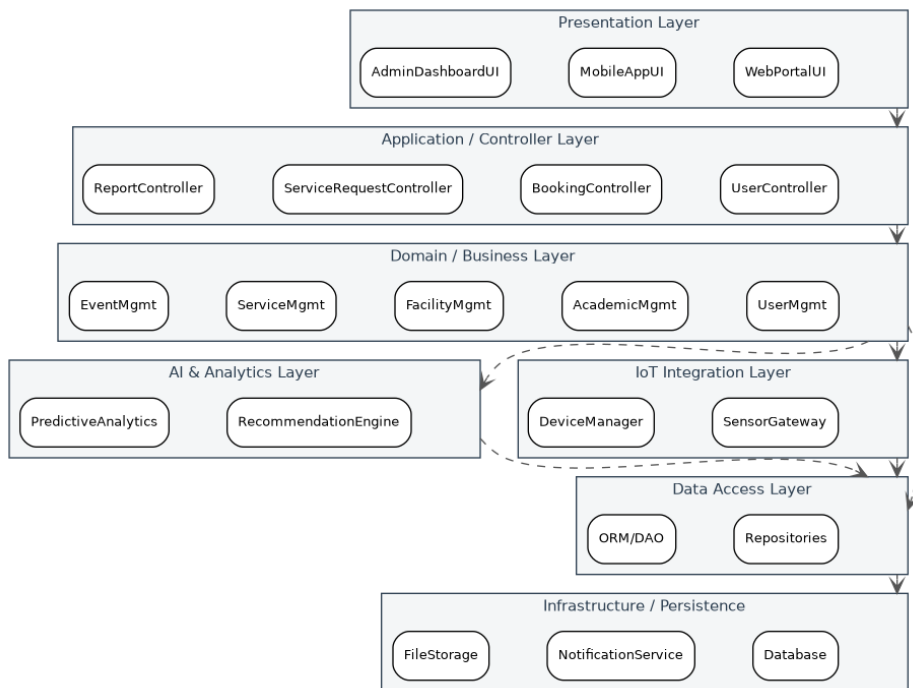


Figure 10.1 — Package Diagram of AISCAMS

11. Component Diagram

The component diagram shows deployable/replaceable software components and the dependency interfaces between them, refining the package diagram into concrete, independently testable units.

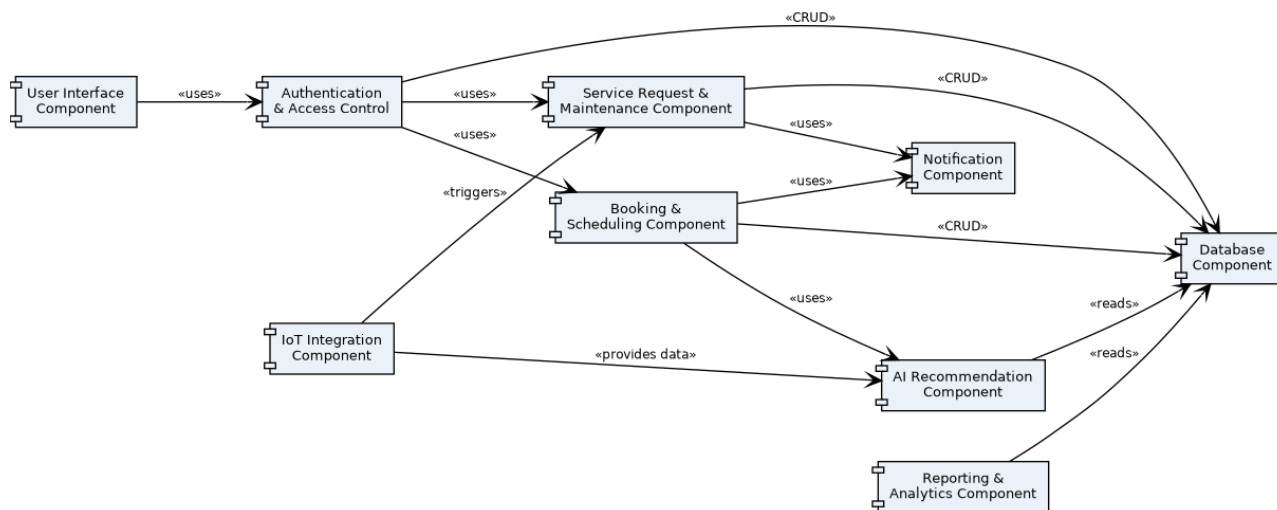


Figure 11.1 — Component Diagram of AISCaMS

12. Deployment Diagram

The deployment diagram maps software components onto physical/virtual nodes: client devices, a cloud/on-premise application server, a dedicated AI server, a database server, an IoT gateway aggregating campus sensor data, and a notification server.

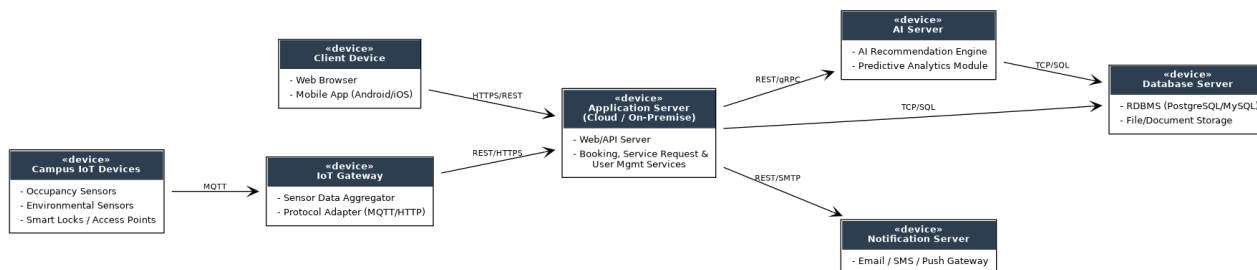


Figure 12.1 — Deployment Diagram of AISCaMS

13. Object Responsibility Analysis (CRC)

Class-Responsibility-Collaborator (CRC) analysis was used to assign responsibilities before finalising operations, ensuring each class has a single, well-bounded purpose (Single Responsibility).

Class	Key Responsibilities	Collaborators
Booking	Know its time window, status, and owning Facility; confirm/cancel itself.	Student/Faculty, Facility, AIRecommendationEngine, NotificationService
ServiceRequest	Capture issue details; own its lifecycle status; expose status to the requester.	Student/Faculty, MaintenanceTask, NotificationService
MaintenanceTask	Track assignment and resolution notes; drive ServiceRequest status transitions.	MaintenanceStaff, ServiceRequest, NotificationService
AIRecommendationEngine	Filter and rank facilities/resources; analyse IoT-derived usage patterns.	Facility, IoTDevice, Booking

Facility	Know its own availability and capacity; accept/deny reservation.	Booking, IoTDevice
NotificationService	Deliver alerts to any User on behalf of any other class (Observer-style).	All entity classes that raise events

14. Design Axioms, Principles, and Design Pattern Justification

14.1 Design Axioms (Suh's Axiomatic Design)

- Independence Axiom — maintained by separating concerns into distinct packages (Booking, ServiceRequest, AI, IoT) so that a change to one functional requirement (e.g., how recommendations are ranked) does not force a change in an unrelated class (e.g., ServiceRequest).
- Information Axiom — the design minimises complexity by keeping each class's interface small and by delegating cross-cutting concerns (recommendation, notification) to dedicated, reusable services rather than duplicating logic across entity classes.

14.2 Object-Oriented Design Principles Applied

Principle	Application in AISCaMS
Encapsulation	Attributes such as ServiceRequest.status and Facility.status are private/protected and mutated only through class-owned operations (e.g., updateStatus()), preventing invalid external state changes.
Abstraction	User and Facility are abstract classes exposing only the operations relevant to client code, hiding role-/type-specific implementation details.
Inheritance	Student, Faculty, Administrator, SecurityPersonnel, and MaintenanceStaff inherit common identity behaviour from User; Classroom, Laboratory, and ParkingArea inherit from Facility.
Polymorphism	Calling dashboard_summary()/login() on a User reference executes the correct role-specific behaviour at runtime without the caller knowing the concrete subclass.
High Cohesion	Each class (e.g., MaintenanceTask) groups only tightly related data and behaviour concerning one responsibility.
Low Coupling	Layered packages and interface-mediated dependencies (e.g., Booking depends on the AIRecommendationEngine's public operation, not its internals) minimise ripple effects of change.

14.3 Design Patterns Selected and Justified

Pattern	Where Applied	Justification
Singleton	AIRecommendationEngine, NotificationService	Exactly one shared engine/service instance should coordinate recommendations and outbound notifications system-wide, avoiding duplicated state or conflicting model instances.
Observer	NotificationService notifying Users on ServiceRequest/Booking state changes	Multiple, loosely-coupled recipients must react to state changes in entity classes without those entity classes knowing notification-delivery details.
State	ServiceRequest / MaintenanceTask lifecycle	The object's allowed operations depend on its current status (e.g., completeTask() is only meaningful in IN_PROGRESS); the State pattern avoids large conditional blocks and centralises transition logic.
Strategy	AIRecommendationEngine.recommendFacility()	Different ranking strategies (capacity-first, proximity-first, usage-pattern-first) can be swapped without changing the calling BookingController.
Factory Method	User/Facility subtype instantiation during account or facility provisioning	Decouples the Administrator-facing account/facility-creation workflow from the concrete subclass being constructed, easing future addition of new roles or facility types.

15. Assumptions and Constraints

15.1 Assumptions

- All campus users are provisioned with a unique institutional account prior to system use (no public self-registration).
- IoT devices are pre-installed, network-connected, and periodically push readings; device provisioning/firmware management is out of scope.
- A single academic calendar governs scheduling; multi-campus or multi-calendar support is not required in this iteration.
- The AI Recommendation Engine operates on historical usage data already present in the system (no external data feeds assumed).
- Payment processing (e.g., for paid facility bookings) is out of scope; all bookings are treated as free-of-charge reservations.

15.2 Constraints

- The system boundary excludes third-party academic ERP systems; AISCAMS integrates with them only through the Notification and Reporting components, not direct data ownership.
- The design must remain technology-agnostic at the model level (no diagram assumes a specific programming language or cloud vendor).
- Real-time IoT data ingestion must not block or degrade the responsiveness of user-facing booking/request transactions (handled via the separate IoT Integration package).
- All UML models must remain mutually consistent: operations on the Class Diagram must be sufficient to support every message shown in the Sequence Diagrams.

16. Requirement-to-Use-Case/Class Traceability Matrix

Req. ID	Requirement	Use Case(s)	Class(es)
R1	Role-based access for all actor types	UC-01	User, Student, Faculty, Administrator, SecurityPersonnel, MaintenanceStaff
R2	Manage academic schedules & classroom bookings	UC-02, UC-03, UC-09	AcademicSchedule, Booking, Facility, Classroom
R3	AI-based facility/resource recommendation	UC-10	AIRecommendationEngine, Facility
R4	Faculty attendance & class management	UC-07, UC-08	Attendance, Course
R5	Campus event management	UC-04	Event
R6	Service request lifecycle & maintenance tracking	UC-06, UC-16	ServiceRequest, MaintenanceTask, MaintenanceStaff
R7	IoT-based infrastructure monitoring	UC-11	IoTDevice, SmartSensor, AIRecommendationEngine
R8	Notifications on status changes	UC-17	NotificationService
R9	Administrative reporting & analytics	UC-13	Report, Administrator
R10	Campus security & parking management	UC-14, UC-15	SecurityPersonnel, ParkingArea

17. Implementation / Prototype and Results

A lightweight Python prototype was implemented to validate the core object-oriented design decisions on a representative slice of the system: the User inheritance hierarchy, the Singleton AIRecommendationEngine and NotificationService, and the State-driven ServiceRequest lifecycle. The full source is listed below, followed by captured console output from a sample run.

17.1 Prototype Source Code

```
"""
AISCams - Minimal Prototype
Demonstrates core OOAD concepts: inheritance, encapsulation, polymorphism,
Singleton pattern, Observer pattern, and State pattern behaviour for a
slice of the Smart Campus Management System (Booking + Service Request flow).
"""

from __future__ import annotations
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from datetime import datetime
from enum import Enum, auto
from typing import List

# ----- User hierarchy (Inheritance + Polymorphism) -----
class User(ABC):
    def __init__(self, user_id: str, name: str):
        self._user_id = user_id
        self._name = name

    @abstractmethod
    def dashboard_summary(self) -> str:
        ...

    def __str__(self):
        return f"{self.__class__.__name__}({self._name})"

class Student(User):
    def __init__(self, user_id, name, reg_no):
        super().__init__(user_id, name)
        self.reg_no = reg_no

    def dashboard_summary(self) -> str:
        return f"Student dashboard for {self._name} [{self.reg_no}]"

class MaintenanceStaff(User):
    def __init__(self, user_id, name, specialization):
        super().__init__(user_id, name)
        self.specialization = specialization

    def dashboard_summary(self) -> str:
        return f"Maintenance dashboard for {self._name} ({self.specialization})"

# ----- Observer pattern: NotificationService -----
```

```

class NotificationService:
    _instance = None # Singleton

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._log = []
        return cls._instance

    def notify(self, user: User, message: str):
        entry = f"[{datetime.now().strftime('%H:%M:%S')}] -> {user}: {message}"
        self._log.append(entry)
        print(entry)

# ----- Facility (simplified) -----
class Facility:
    def __init__(self, facility_id, name, capacity):
        self.facility_id = facility_id
        self.name = name
        self.capacity = capacity
        self._available = True

    def check_availability(self) -> bool:
        return self._available

    def reserve(self):
        self._available = False

# ----- AI Recommendation Engine (Singleton + Strategy-like) -----
class AIRecommendationEngine:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def recommend_facility(self, facilities: List[Facility], min_capacity: int) -> Facility | None:
        candidates = [f for f in facilities if f.check_availability() and f.capacity >= min_capacity]
        return candidates[0] if candidates else None

# ----- Service Request lifecycle (State pattern) -----
class RequestStatus(Enum):
    SUBMITTED = auto()
    ASSIGNED = auto()
    IN_PROGRESS = auto()
    RESOLVED = auto()
    CLOSED = auto()

@dataclass

```

```

class ServiceRequest:
    request_id: str
    raised_by: Student
    description: str
    status: RequestStatus = RequestStatus.SUBMITTED
    assigned_staff: MaintenanceStaff | None = None

    def assign(self, staff: MaintenanceStaff):
        self.assigned_staff = staff
        self.status = RequestStatus.ASSIGNED
        NotificationService().notify(staff, f"New task assigned: {self.request_id}")

    def start_progress(self):
        self.status = RequestStatus.IN_PROGRESS

    def resolve(self):
        self.status = RequestStatus.RESOLVED
        NotificationService().notify(self.raised_by, f"Your request {self.request_id} has been resolved.")

    def close(self):
        self.status = RequestStatus.CLOSED

# ----- Demo run -----
if __name__ == "__main__":
    alice = Student("S001", "Alice Rao", "REG2026CS045")
    bob = MaintenanceStaff("M001", "Bob Kumar", "Electrical")

    print(alice.dashboard_summary())
    print(bob.dashboard_summary())
    print("-" * 60)

    facilities = [Facility("F1", "Room 204", 40), Facility("F2", "Lab 3", 25)]
    engine = AIRecommendationEngine()
    chosen = engine.recommend_facility(facilities, min_capacity=20)
    print(f"AI recommended facility: {chosen.name} (capacity {chosen.capacity})")
    chosen.reserve()
    print(f"{chosen.name} available after booking? {chosen.check_availability()}")
    print("-" * 60)

    req = ServiceRequest("SR-1001", alice, "Projector not working in Room 204")
    print(f"Request created. Status: {req.status.name}")
    req.assign(bob)
    print(f"Status: {req.status.name}")
    req.start_progress()
    print(f"Status: {req.status.name}")
    req.resolve()
    print(f"Status: {req.status.name}")
    req.close()
    print(f"Final status: {req.status.name}")

```

17.2 Sample Execution Output

Student dashboard for Alice Rao [REG2026CS045]

Maintenance dashboard for Bob Kumar (Electrical)

```
-----  
AI recommended facility: Room 204 (capacity 40)  
Room 204 available after booking? False  
-----
```

```
Request created. Status: SUBMITTED  
[07:50:48] -> MaintenanceStaff(Bob Kumar): New task assigned: SR-1001  
Status: ASSIGNED  
Status: IN_PROGRESS  
[07:50:48] -> Student(Alice Rao): Your request SR-1001 has been resolved.  
Status: RESOLVED  
Final status: CLOSED
```

The output confirms that: (a) polymorphic `dashboard_summary()` calls `resolve` to the correct subclass implementation; (b) both Singleton services (`AIRecommendationEngine`, `NotificationService`) coordinate shared state correctly across calls; and (c) the `ServiceRequest` object moves strictly through the `SUBMITTED` → `ASSIGNED` → `IN_PROGRESS` → `RESOLVED` → `CLOSED` sequence defined in the State Transition Diagram, with no illegal transitions possible from the public interface — validating the design's consistency end-to-end.

18. Reflection on Design Decisions and Learning Outcomes

Working through AISCaMS end-to-end reinforced that Object-Oriented Analysis and Design is fundamentally an exercise in disciplined abstraction rather than diagram production. The hardest design decision was drawing the boundary between the AI Recommendation Engine as a Control-class service versus modelling it as a passive Entity attribute of Facility; treating it as an independent Singleton service kept the Facility class cohesive and made the recommendation logic swappable (Strategy pattern) without perturbing the rest of the model — a direct, practical illustration of the Independence Axiom and the Open/Closed Principle.

A second key insight came from building the State Transition Diagram before finalising the Class Diagram's operations: it exposed that `ServiceRequest` needed an explicit `ON_HOLD` and `REOPENED` state that had been missed in the first pass of the Sequence Diagram, demonstrating why UML models must be developed iteratively and cross-checked for consistency rather than sequentially. Implementing the small Python prototype was similarly valuable — it surfaced an initial coupling issue between `ServiceRequest` and `MaintenanceStaff` that was resolved by introducing the `NotificationService` as an Observer-style intermediary.

In terms of Sustainable Development Goals, the system directly advances SDG 4 (Quality Education) by ensuring classrooms, laboratories, and digital learning resources are allocated efficiently to students and faculty; SDG 9 (Industry, Innovation and Infrastructure) through its use of IoT sensing and AI-driven optimisation of physical infrastructure; and SDG 11 (Sustainable Cities and Communities) by reducing energy and space wastage on campus through occupancy-aware facility management. Overall, the assignment strengthened practical fluency in translating an informal scenario into a consistent, traceable, and pattern-justified object-oriented model.

19. GitHub Upload